

Informe de desarrollo

Unidad 2 - Sistema de gestión con Lista, Cola y Pila

Campo	Información
Universidad	Universidad de Cartagena
Asignatura	Estructuras de datos
Actividad	Actividad Unidad 2 - Ejercicio 4: Restaurante
Estudiante	[Nombre completo del estudiante]
Código	[Código del estudiante]
Programa académico	Ingeniería del software
Docente	John Carlos Arrieta Arrieta
Fecha de entrega	10 de junio de 2026

Nota: reemplace los campos entre corchetes antes de entregar el PDF final.

Logros de aprendizaje

Durante el desarrollo de esta actividad comprendí cómo las estructuras de datos permiten organizar la información de acuerdo con la necesidad funcional del sistema. En el caso del restaurante, cada pedido entra al sistema, queda registrado en una lista general, espera su turno en una cola de pendientes y, cuando se procesa, pasa a una pila de historial.

Aprendí que una lista permite conservar todos los pedidos registrados y recorrerlos cuando se requiere consultar el estado general del restaurante. La cola se ajusta al principio FIFO, porque el primer pedido que ingresa debe ser el primero en atenderse. La pila trabaja bajo el principio LIFO, por lo que el último pedido procesado es el primero que puede deshacerse si se cometió un error.

También fortalecí la comprensión de los nodos enlazados. Cada nodo guarda un dato y referencias hacia otros nodos. En la lista doblemente enlazada se utilizan referencias al nodo anterior y al siguiente para recorrer hacia adelante y hacia atrás. En la cola se mantiene una referencia al frente y al final, mientras que en la pila se controla la cima.

Una dificultad importante fue mantener coherencia entre las estructuras: cuando un pedido se registra debe entrar tanto a la lista como a la cola; cuando se procesa debe salir de la cola y entrar al historial; y cuando se deshace debe salir del historial y regresar a la cola. Esto fortaleció mi lógica de programación y mi manejo del flujo de datos.

En programación orientada a objetos apliqué encapsulamiento mediante atributos privados, constructores, métodos getters y setters, método toString y método equals para comparar pedidos por número de pedido. Además, el sistema se modularizó en varias clases para separar responsabilidades.

Sobre validaciones y manejo de errores, aprendí a verificar datos vacíos, números inválidos, pedidos duplicados y operaciones imposibles, como procesar cuando la cola está vacía o deshacer cuando no existe historial. También se fortaleció el uso de Git porque el proyecto debe construirse con commits progresivos y descriptivos.

Informe de desarrollo

Descripción funcional del sistema

La aplicación resuelve la administración básica de pedidos en un restaurante. Permite registrar pedidos, enviarlos a una cola de pendientes, procesarlos por orden de llegada, guardar los pedidos procesados en un historial, buscar pedidos por número, cancelar pedidos pendientes, deshacer el último procesamiento y visualizar cantidades generales.

Operación	Descripción
Registrar pedido	Crea un pedido con número, cliente, descripción, total y estado PENDIENTE.
Ver registrados	Muestra todos los pedidos guardados en la lista general.
Ver pendientes	Muestra la cola de pedidos pendientes y el siguiente pedido a procesar.
Procesar	Desencola el primer pedido pendiente, cambia su estado a PROCESADO y lo apila en el historial.
Buscar	Busca un pedido en la lista general usando equals por número de pedido.
Cancelar	Retira un pedido pendiente usando una cola auxiliar y cambia su estado a CANCELADO.
Deshacer	Desapila el último procesado, cambia su estado a PENDIENTE y lo reencola.

Componentes desarrollados

Clase	Responsabilidad
-------	-----------------

Nodo	Representa una unidad enlazada con dato, siguiente y anterior.
Lista	Almacena todos los pedidos registrados y permite agregar, eliminar, buscar, contar y recorrer.
Cola	Administra los pedidos pendientes bajo el principio FIFO.
Pila	Guarda el historial de pedidos procesados bajo el principio LIFO.
Pedido	Entidad principal del restaurante. Su identificador es el número de pedido.
SistemaRestaurante	Clase de ejecución con menú, validaciones y flujo del sistema.

Explicación de las estructuras de datos usadas

Lista: se implementó como una lista doblemente enlazada. Agrega pedidos al final, permite agregar al inicio o en una posición, eliminar elementos y recorrer en ambos sentidos. En el sistema resuelve el requisito de conservar el registro completo de pedidos.

Cola: se implementó con referencia al frente y al final. Al encolar se agrega al final y al desencolar se retira desde el frente. Así se garantiza que los pedidos sean atendidos en orden de llegada.

Pila: se implementó con referencia a la cima. Al apilar se agrega un pedido encima del historial y al desapilar se retira el último procesado. Esto permite deshacer la operación más reciente.

Fragmentos clave de código

```
// Registro obligatorio
listaGeneral.agregar(pedido);
colaPendientes.encolar(pedido);

// Procesamiento obligatorio
Pedido procesado = (Pedido) colaPendientes.desencolar();
procesado.setEstado("PROCESADO");
historialProcesados.apilar(procesado);

// Deshacer procesamiento
Pedido ultimo = (Pedido) historialProcesados.desapilar();
ultimo.setEstado("PENDIENTE");
colaPendientes.encolar(ultimo);
```

Flujo del sistema

1. El usuario registra un pedido con número, cliente, descripción y total.
2. El pedido se almacena en la lista general para conservar evidencia completa.
3. El pedido entra a la cola de pendientes para esperar su turno.
4. Al procesar, el sistema desencola el primer pedido pendiente.
5. El pedido procesado cambia su estado a PROCESADO y se apila en el historial.
6. La búsqueda se realiza en la lista usando un objeto auxiliar con el número de pedido.
7. La cancelación de pendientes se realiza con una cola auxiliar, sin eliminar directamente desde el centro de la cola.
8. Al deshacer, el último pedido del historial vuelve a la cola como PENDIENTE.

Diagrama textual del flujo

```
Registro:      Pedido -> Lista general -> Cola de pendientes
Procesamiento: Cola de pendientes -> Pedido procesado -> Pila historial
Deshacer:      Pila historial -> Pedido pendiente -> Cola de pendientes
Cancelacion:    Cola original -> Cola auxiliar, omitiendo el pedido cancelado
```

Evidencia de ejecución

Evidencia de ejecucion por consola

```
===== SISTEMA DE PEDIDOS - RESTAURANTE =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por codigo
7. Cancelar elemento pendiente
8. Deshacer ultimo procesamiento
9. Ver cantidad de elementos
10. Salir
Seleccione una opcion:
--- Registrar pedido ---
Numero de pedido: Nombre del cliente: Descripcion del pedido: Total del pedido: Pedido registrado y enviado a

===== SISTEMA DE PEDIDOS - RESTAURANTE =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por codigo
7. Cancelar elemento pendiente
8. Deshacer ultimo procesamiento
9. Ver cantidad de elementos
10. Salir
Seleccione una opcion:
--- Registrar pedido ---
Numero de pedido: Nombre del cliente: Descripcion del pedido: Total del pedido: Pedido registrado y enviado a

===== SISTEMA DE PEDIDOS - RESTAURANTE =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por codigo
7. Cancelar elemento pendiente
8. Deshacer ultimo procesamiento
9. Ver cantidad de elementos
10. Salir
```

Enlace del repositorio GitHub

[Pegar aquí el enlace público del repositorio GitHub]

El repositorio debe evidenciar commits progresivos, mensajes claros y evolución del desarrollo. En la carpeta docs se anexan comandos sugeridos para crear el historial y subir el proyecto.

Enlace del video de sustentación

[Pegar aquí el enlace del video de sustentación]

El video debe demostrar el funcionamiento de la aplicación y explicar el código, no limitarse a leer este informe.

Conclusiones

El sistema demuestra que una misma entidad puede circular por varias estructuras según su estado dentro del proceso. La lista funciona como memoria general, la cola como mecanismo de atención por orden de llegada y la pila

como historial que permite revertir la última operación. La actividad fortalece la comprensión interna de las estructuras enlazadas y su aplicación en un caso real.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms. MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java. Wiley.

Oracle. (s. f.). The Java Tutorials. Oracle Documentation.